

How Fully Homomorphic Encryption became (somehow) practical

September 2, 2026

Chiara Marcolla

chiara.marcolla@tii.ae

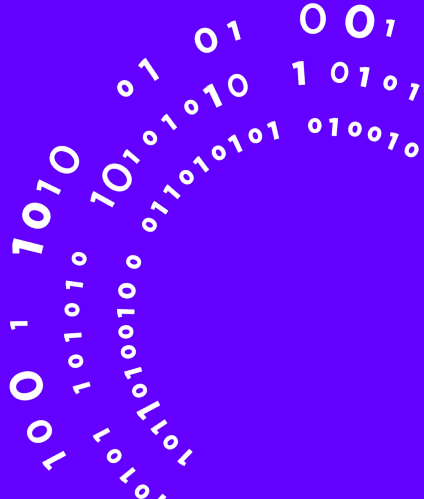
Contents



1. Why fully homomorphic encryption
2. The second generation: managing the error

3. Making it fast: packing and representations
4. Back to history . . .
5. For tomorrow...

Why fully homomorphic encryption



The diagram illustrates the concept of cloud storage and synchronization. On the left, a silhouette of a person sitting at a desk with a laptop is shown. A large, double-headed blue arrow connects the person to a large, light blue cloud on the right. Inside the cloud, there are numerous icons representing different types of files and applications, including:

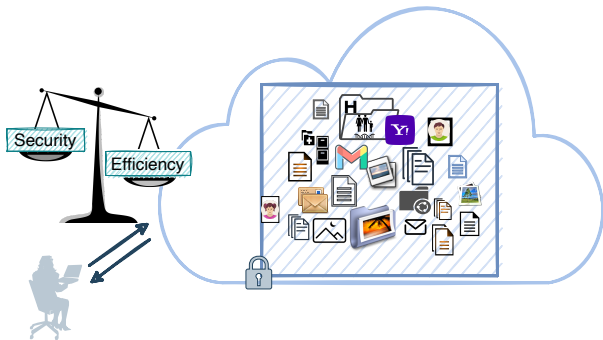
- Google Drive (G logo)
- OneDrive (H logo)
- Dropbox (D logo)
- Various document icons (pages with lines of text)
- Image icons (landscape, portrait, abstract)
- Folder icons (some with labels like 'FOLDER' and 'FOLDER')
- Application icons (e.g., a calendar, a mail icon)

 This visualizes how data is stored in the cloud and can be accessed and synchronized across different devices and applications.

Cloud has **full access** to (sensitive) data: medical data, emails, financial data, pictures, . . .

Why fully homomorphic encryption

Remote **cloud storage** services (such as Dropbox, Google Drive and Google Photo) are used for backups or outsourcing data.

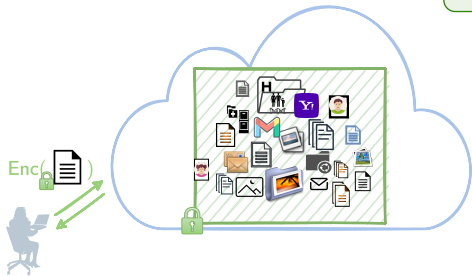


Cloud has **full access** to (sensitive) data $\implies$ **Naïve solution:** encrypt everything.

Why fully homomorphic encryption

Encrypting everything protects the data, but it also makes the data useless to the cloud: nothing can be computed on it any more.

FHE permits computations over encrypted messages

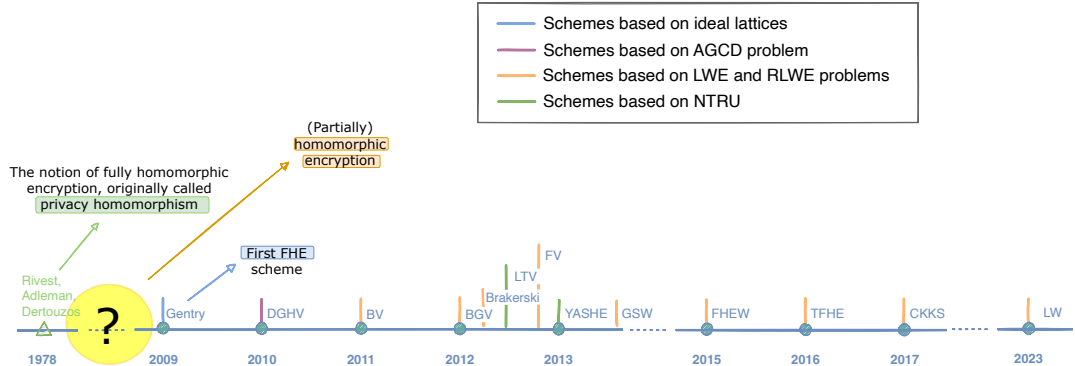


$$\text{Enc}_{\text{pk}}(m_1) \times \text{Enc}_{\text{pk}}(m_2) = \text{Enc}_{\text{pk}}(m_1 \times m_2)$$

$$\text{Enc}_{\text{pk}}(m_1) + \text{Enc}_{\text{pk}}(m_2) = \text{Enc}_{\text{pk}}(m_1 + m_2)$$

Cloud has **full access** to (sensitive) data $\implies$ **Optimal solution:** use FHE.

History...



Rivest, Adleman, Dertouzos On data banks and privacy homomorphisms (1978)

(Partially) homomorphic encryption

- RSA cryptosystem by Rivest, Shamir and Adleman (1978) – homomorphic with respect to **multiplication**
- Goldwasser and Micali (1982) – homomorphic with respect to **bit addition** (XOR)
- ElGamal (1985) – homomorphic with respect to **multiplication**
- Benaloh (1994), Naccache and Stern (1998), Paillier (1999) – homomorphic with respect to **addition**

...almost there ...

- Boneh, Goh and Nissim (2005) an arbitrary number of **additions** and just **one multiplication**, then again an arbitrary number of **additions**.
- Aguilar Melchor, Gaborit and Herranz (2008) proposed a theoretical approach that permits **chaining** several homomorphic schemes in order to have a fixed amount of multiplications and additions.

What is a homomorphic encryption scheme?

A **public key homomorphic encryption scheme** $\mathcal{E}$ is a set of probabilistic polynomial-time algorithms (KeyGen, Enc, Dec, Eval), where (KeyGen, Enc, Dec) are as usual and

- **KeyGen** takes as **input** the security parameter λ and outputs the secret key **sk**, the public key **pk** and a (public) evaluation key **evk**, needed to perform homomorphic operations over ciphertexts;
- **Eval** takes as **input** the evaluation key **evk**, a function f and a t -tuple of ciphertexts $(c_1, \dots, c_t)$, and outputs a **ciphertext** c_f such that

$$c_f = \text{Eval}_{\text{evk}}(f, (c_1, \dots, c_t)) \quad \text{and} \quad \text{Dec}_{\text{sk}}(c_f) = f(m_1, \dots, m_t).$$

Note

The ciphertext c_f is equivalent to $c = \text{Enc}_{\text{pk}}(f(m_1, \dots, m_t))$, i.e. they decrypt to the same plaintext, but they are **different in their construction** (e.g. they may have different noise).

Questions for Homomorphic Encryption Scheme

1 Which functions can the scheme evaluate?

Definition ($\mathcal{F}$ -homomorphic scheme)

The scheme $\mathcal{E}$ is $\mathcal{F}$ -homomorphic if there exists Eval such that $\text{Dec}_{\text{sk}}(\text{Eval}_{\text{evk}}(f, c_1, \dots, c_t)) = f(m_1, \dots, m_t)$ for all $f \in \mathcal{F}$, where $c_i = \text{Enc}_{\text{pk}}(m_i)$.

Questions for Homomorphic Encryption Scheme

- 1 Which functions can the scheme evaluate?
- 2 How much does the ciphertext grow after each homomorphic operation?

Definition ($\mathcal{F}$ -homomorphic scheme)

The scheme $\mathcal{E}$ is $\mathcal{F}$ -homomorphic if there exists Eval such that $\text{Dec}_{\text{sk}}(\text{Eval}_{\text{evk}}(f, c_1, \dots, c_t)) = f(m_1, \dots, m_t)$ for all $f \in \mathcal{F}$, where $c_i = \text{Enc}_{\text{pk}}(m_i)$.

Definition (compact ciphertext)

A ciphertext is compact if the bound of the bit-length growth is independent of the complexity of the homomorphic operations or functions being evaluated.

Fully homomorphic and its weaker cousin

Definition (FHE scheme)

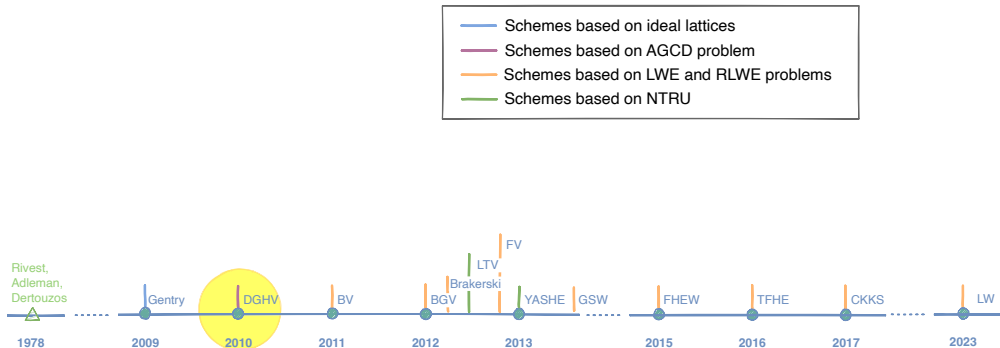
A **Fully Homomorphic Encryption (FHE) scheme** $\mathcal{E}$ is an encryption scheme where the ciphertexts are **compact**, and the scheme is $\mathcal{F}$ -homomorphic, where $\mathcal{F}$ is the set of **all** the (efficiently computable) **functions**.

For the sake of history:

Definition (SHE scheme (Gentry 2009))

A **Somewhat Homomorphic Encryption (SHE) scheme** is a scheme that is $\mathcal{F}$ -homomorphic for a limited class $\mathcal{F}$, e.g. capable of evaluating “**low-degree**” multivariate polynomials homomorphically. In an SHE scheme, the compactness of ciphertexts could be violated.

History...



A scheme small enough to fit on one slide

Let us look at the simplest scheme of the family, DGHV over the integers.

- **Encryption:** the message $m \in \{0, 1\}$

$$c = m + 2e + pq$$

where

- ★ $p \in \mathbb{Z}$ odd is the secret key
- ★ $q \in \mathbb{Z}$ such that $q \gg p$
- ★ $e \in \mathbb{Z}$ is a *small error*

- **Decryption:**

$$m = (c \bmod p) \bmod 2$$

Both operations, and the price for them

$$c = m + 2e + pq \quad c' = m' + 2e' + pq'$$

- Addition: $c + c' = m + m' + 2(e + e') + p(q + q')$
- Multiplication: $c \cdot c' = m \cdot m' + 2(2e \cdot e' + \dots) + p(pqq' + \dots)$

Both operations, and the price for them

$$c = m + 2e + pq \quad c' = m' + 2e' + pq'$$

- Addition: $c + c' = m + m' + 2(e + e') + p(q + q')$
- Multiplication: $c \cdot c' = m \cdot m' + 2(2e \cdot e' + \dots) + p(pqq' + \dots)$

Decryption is correct only if the error is not *too* big.

$$\text{Dec}(c \cdot c') = (c \cdot c' \bmod p) \bmod 2$$

The error grows **linearly** with the number of **additions** and **exponentially** with the number of **multiplications**.

From SHE to FHE: bootstrapping

How can we reduce the error?

From SHE to FHE: bootstrapping

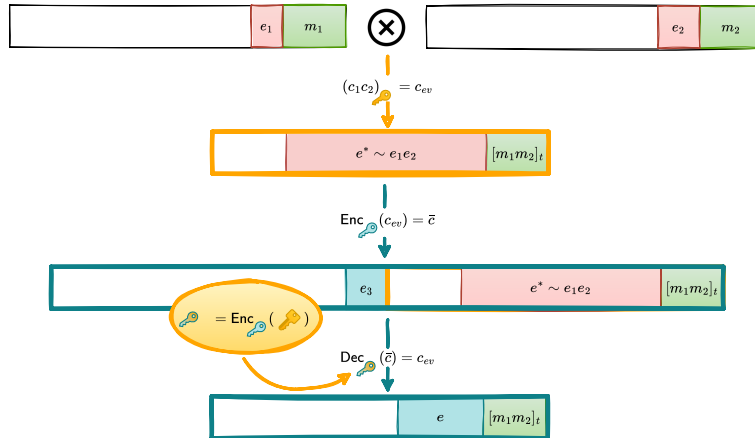
How can we reduce the error?

Gentry's observation (2009)

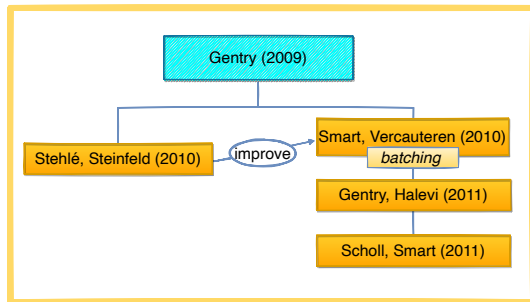
An SHE scheme with sufficient homomorphic evaluation capacity can be transformed into an FHE scheme by using the **bootstrapping technique**.

The key idea: **decrypt the ciphertext homomorphically**.

Bootstrapping in one picture



History... first generation



Ideal lattice-based FHE schemes

- + First fully homomorphic scheme
- Difficult to implement efficiently
- 2016: efficient attack [1]



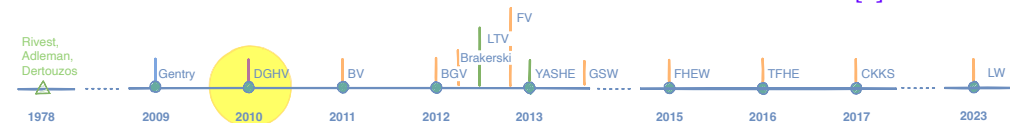
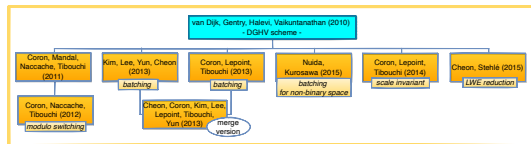
[1] Cramer, Ducas, Peikert, Regev - *Recovering short generators of principal ideals in cyclotomic rings* (2016)

History... first generation

AGCD-based FHE schemes

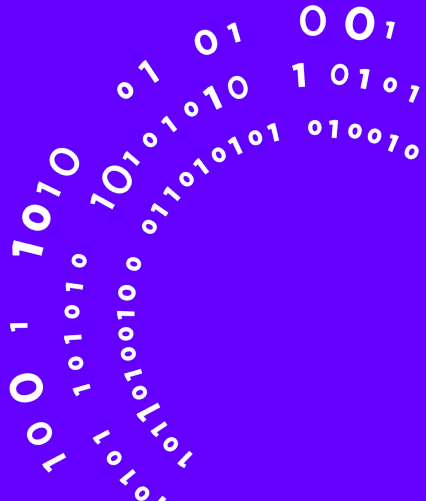
Approximate - Greatest Common Divisor

- + Very simple
- High computational complexity
- Large public key size
- 2012: efficient attack [2]



[2] Chen, Nguyen - *Faster algorithms for approximate common divisors: Breaking fully-homomorphic-encryption challenges over the integers* (2012)

The second generation: managing the error





- + New method to manage the error: modulus switching $\Rightarrow$ leveled version
- + Efficient packing
- Slow bootstrapping
- No non-arithmetic functions



A **lattice** $\mathcal{L}$ generated by $\mathbf{b}_1, \dots, \mathbf{b}_n \in \mathbb{R}^m$ is the set of linear combinations of $\mathbf{b}_1, \dots, \mathbf{b}_n$ with coefficients in $\mathbb{Z}$,

$$\mathcal{L}(\mathbf{b}_1, \dots, \mathbf{b}_n) = \{\mathbf{x} \in \mathbb{R}^m : \mathbf{x} = a_1\mathbf{b}_1 + a_2\mathbf{b}_2 + \dots + a_n\mathbf{b}_n \text{ where } a_i \in \mathbb{Z}\}.$$

A **lattice** $\mathcal{L}$ generated by $\mathbf{b}_1, \dots, \mathbf{b}_n \in \mathbb{R}^m$ is the set of linear combinations of $\mathbf{b}_1, \dots, \mathbf{b}_n$ with coefficients in $\mathbb{Z}$,

$$\mathcal{L}(\mathbf{b}_1, \dots, \mathbf{b}_n) = \{\mathbf{x} \in \mathbb{R}^m : \mathbf{x} = a_1 \mathbf{b}_1 + a_2 \mathbf{b}_2 + \dots + a_n \mathbf{b}_n \text{ where } a_i \in \mathbb{Z}\}.$$

Let q be a positive integer, a **q -ary lattice** is a lattice $\mathcal{L}_q(A) \subseteq \mathbb{Z}^m$ defined by $A \in \mathbb{Z}_q^{m \times n}$:

$$\mathcal{L}_q(A) := \{\mathbf{x} \in \mathbb{Z}^m : \exists \mathbf{s} \in \mathbb{Z}^n \text{ such that } A\mathbf{s} = \mathbf{x} \bmod q\}.$$

LWE Problem - as a lattice problem

Definition (*Search* LWE Problem (Regev))

Given $\mathbf{b} \in \mathbb{Z}_q^m$ and $A \in (\mathbb{Z}_q)^{m \times n}$, find an *unknown* vector $\mathbf{s} \in \mathbb{Z}_q^n$ such that

$$A\mathbf{s} + \mathbf{e} = \mathbf{b} \bmod q$$

where $\mathbf{e} \in \mathbb{Z}_q^m$ is *small* random error.

Bounded Distance Decoding Problem

Let $A \in (\mathbb{Z}_q)^{m \times n}$, then

$$\mathcal{L}_q(A) := \{\mathbf{x} \in \mathbb{Z}^m : \exists \mathbf{s} \in \mathbb{Z}^n \text{ s.t. } A\mathbf{s} = \mathbf{x} \bmod q\},$$

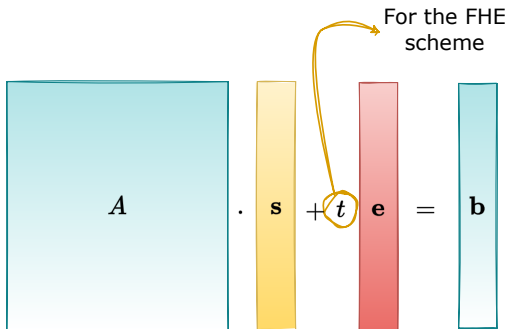
The **Bounded Distance Decoding (BDD)** problem asks to find $\mathbf{x} = A\mathbf{s} \in \mathcal{L}$ closest to the target $\mathbf{t} = \mathbf{b}$ with the promise that $\|\mathbf{t} - \mathbf{x}\| = \|\mathbf{e}\| \leq \alpha$, where $\alpha \ll \lambda_1(\mathcal{L})$.

From the LWE problem to an LWE-based scheme

Recall – LWE problem

Given $\mathbf{b} \in \mathbb{Z}_q^m$, $A \in (\mathbb{Z}_q)^{m \times n}$, and $\mathbf{e} \in \mathbb{Z}_q^m$ a *small* random error, find $\mathbf{s} \in \mathbb{Z}_q^n$ such that

$$A\mathbf{s} + \mathbf{e} = \mathbf{b} \bmod q.$$



- The **secret key** $\mathbf{s} \in \chi_s^n$
- The **public key** $(A, A\mathbf{s} + t\mathbf{e} = \mathbf{b})$

RLWE Problem

The m -th cyclotomic polynomial $\Phi_m(x)$, for any positive integer m , is the **unique irreducible** polynomial with **integer coefficients** that is a divisor of $x^m - 1$ and is *not* a divisor of $x^k - 1$ for any $k < m$:

$$\Phi_m(x) = \prod_{(j,m)=1} (x - \xi^j),$$

where ξ is a primitive root m -th root of unity (i.e., $\xi^m = 1$ and $\xi^k \neq 1$ for $k = 1, 2, \dots, m-1$).

Definition (Search RLWE Problem)

Let $\mathcal{R}_q = \mathbb{Z}_q[x] / \langle \Phi_m(x) \rangle$, where $\Phi_m(x)$ is the m -th cyclotomic polynomial.

Find an *unknown* polynomial $s \in \mathcal{R}_q$ given access to many independent RLWE samples

$$\{(a_i, b_i = a_i \cdot s + e_i) \in \mathcal{R}_q \times \mathcal{R}_q\}_{i=1, \dots, k}$$

where $a_i \in \mathcal{R}_q$, and $e_i \in \mathcal{R}_q$ are *small* random error polynomials.

From LWE to RLWE

Working with matrices over $\mathbb{Z}_q$ is expensive: a public key is a full matrix, and matrix-vector products cost $O(n^2)$.

Ring-LWE replaces vectors by polynomials in the ring

$$\mathcal{R}_q = \mathbb{Z}_q[x] / \langle \Phi_m(x) \rangle, \quad \text{typically with } \Phi_m(x) = x^n + 1, \quad m = 2n = 2^k,$$

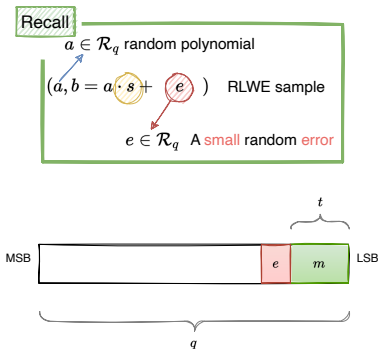
and an RLWE sample has the form

$$b = a \cdot s + e \in \mathcal{R}_q, \quad a \leftarrow \mathcal{U}_q, \quad s \leftarrow \chi_s, \quad e \leftarrow \chi_e.$$

- ✓ a public key consists of **two ring elements** instead of a matrix;
- ✓ one ring multiplication can be implemented in $O(n \log n)$ – we come back to this;
- ✓ SIMD packing allows **many messages to be packed into one polynomial** – we come back to this.

Modern FHE schemes are typically built on RLWE or its generalisation, GLWE.

The BGV scheme on one slide



Leveled BGV scheme on RLWE

$$\Phi_m(x) = x^n + 1 \text{ where } n = 2^\kappa$$

$$m \in \mathbb{Z}_t[x]/\langle x^n + 1 \rangle \quad \mathbf{c} \in (\mathbb{Z}_q[x]/\langle x^n + 1 \rangle)^2$$

$$\begin{array}{ccc} \text{sk} = s & & \text{pk} = (a, b) \equiv (a, -a \cdot s + te) \\ \downarrow \chi_s & & \downarrow \mathcal{U}_q \quad \downarrow \chi_e \end{array}$$

$\text{Enc}(m)$

$$\mathbf{c} = (c_0, c_1) \equiv (b \cdot u + te_0 + \boxed{m}, a \cdot u + te_1) \pmod{q}$$

$\xrightarrow{-a \cdot s + te}$

$\text{Dec}(\mathbf{c})$

$$c_0 + c_1 \cdot s \equiv m + tE \pmod{q}$$

$$m + tE \equiv m \pmod{t} \text{ where } E = e \cdot u + s \cdot e_1 + e_0$$

Homomorphic operations

$\mathbf{c} = (c_0, c_1)$ can be see as a polynomial $c(x) \implies \text{Dec}(\mathbf{c}) = [[c(s)]_q]_t = [[c_0 + c_1 s]_q]_t$.

Let $\mathbf{c} = (c_0, c_1)$ and $\mathbf{c}' = (c'_0, c'_1)$ be two ciphertexts.

- **Add**($\mathbf{c}, \mathbf{c}'$): output $([c_0 + c'_0]_q, [c_1 + c'_1]_q)$ – still two components.
- **Mul**($\mathbf{c}, \mathbf{c}'$): output $([c_0 c'_0]_q, [c_0 c'_1 + c_1 c'_0]_q, [c_1 c'_1]_q)$ – **three** components.

Homomorphic operations

$\mathbf{c} = (c_0, c_1)$ can be see as a polynomial $c(x) \implies \text{Dec}(\mathbf{c}) = [[c(s)]_q]_t = [[c_0 + c_1 s]_q]_t$.

Let $\mathbf{c} = (c_0, c_1)$ and $\mathbf{c}' = (c'_0, c'_1)$ be two ciphertexts.

- **Add**($\mathbf{c}, \mathbf{c}'$): output $([c_0 + c'_0]_q, [c_1 + c'_1]_q)$ – still two components.
- **Mul**($\mathbf{c}, \mathbf{c}'$): output $([c_0 c'_0]_q, [c_0 c'_1 + c_1 c'_0]_q, [c_1 c'_1]_q)$ – **three** components.

Multiplication creates **two problems at once**:

- ★ the **ciphertext grows**
 $\implies$ **re-linearization/key switching**;
- ★ the **error grows**, and multiplicatively;
 $\implies$ **modulus switching**.

Focus on the multiplication

Let $\mathbf{c} = (c_0, c_1)$ and $\mathbf{c}' = (c'_0, c'_1)$, then

$$\mathbf{d} = \mathbf{c} \cdot \mathbf{c}' = (d_0, d_1, d_2) = (c_0 \cdot c'_0, c_0 \cdot c'_1 + c_1 \cdot c'_0, c_1 \cdot c'_1)$$

represents the **product of the two polynomials $c(x)$ and $c'(x)$** .

To decrypt $\mathbf{d}$, we have to calculate $[[d(s) = d_0 + d_1 \cdot s + d_2 \cdot s^2]_q]_t$.

The **extended ciphertext** $(d_0, d_1, d_2) \implies$ decrypted using an **extended secret key** $(1, s, s^2)$.

The inconvenience is that every multiplication expands the decryption key.

Key switching – intuitively

Instead of using this *special* decryption, we use a **relinearization procedure** (called **key switching**) to convert the ciphertext $\mathbf{d} = (d_0, d_1, d_2) \in \mathcal{R}_q^3$ back to a ciphertext $\bar{\mathbf{c}} = (\bar{c}_0, \bar{c}_1) \in \mathcal{R}_q^2$.

convert $d_2 s^2 \implies \tilde{c}_0 + \tilde{c}_1 s$ using *somehow* the encryption of s^2 under s .

Indeed,

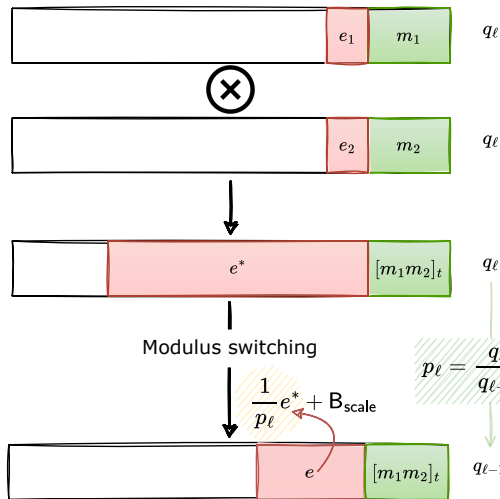
$$\begin{aligned} \text{Enc}_s(s^2) = (\beta, \alpha) &= (-us + te + s^2, u + te_1) \\ &\approx (\alpha s + s^2, -\alpha). \end{aligned}$$

Thus, $s^2 \approx \beta - \alpha s$ and then we have a *normal* ciphertext:

$$d_0 + d_1 s + d_2 s^2 = \textcolor{brown}{d}_0 + \textcolor{teal}{d}_1 s + d_2(\textcolor{brown}{\beta} - \textcolor{teal}{\alpha} s) = \bar{c}_0 + \bar{c}_1 s$$

-
- [1] Brakerski, Vaikuntanathan - *Fully Homomorphic Encryption from Ring-LWE and Security for Key Dependent Messages* (2011)
 [2] Gentry, Halevi, Smart - *Homomorphic Evaluation of the AES Circuit* (2012)

A new method for managing the error



Any time that we do a homomorphic operation, the error increase $\implies$ Brakerski, Gentry, and Vaikunthanathan proposed a new method for managing the error in FHE.

The basic idea is to **divide the error** by some p . Thus, we have to divide the ciphertext by p

$$c \bmod q \implies c' = \frac{c}{p} \bmod \frac{q}{p}$$

such that $m = \text{Dec}(c) = \text{Dec}(c')$.

Levels

Every time that we divide by p , we go down by **one level**

The consequence: leveled fully homomorphic encryption



We fix a chain of moduli $q_{L-1} = \prod_{j=0}^{L-1} p_j$ and we spend one level per multiplication.

Definition (leveled fully homomorphic)

A **leveled fully homomorphic** scheme is a scheme that **supports** the evaluation of circuits of a **specific depth**: it is $\mathcal{F}$ -homomorphic, where $\mathcal{F}$ is the set of all functions of some specific degree, and the bound on the length of the ciphertext is independent of such degree.

The consequence: leveled fully homomorphic encryption



We fix a chain of moduli $q_{L-1} = \prod_{j=0}^{L-1} p_j$ and we spend one level per multiplication.

Definition (leveled fully homomorphic)

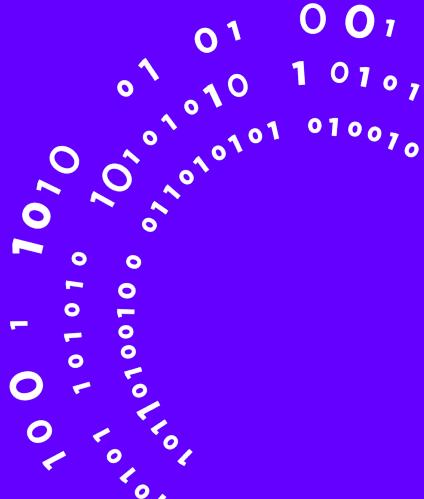
A **leveled fully homomorphic** scheme is a scheme that **supports** the evaluation of circuits of a **specific depth**: it is $\mathcal{F}$ -homomorphic, where $\mathcal{F}$ is the set of all functions of some specific degree, and the bound on the length of the ciphertext is independent of such degree.

The **depth** becomes a **setup parameter of the scheme**.

Why this changed everything

A leveled FHE scheme **avoids** the computationally **expensive bootstrapping** technique. This made the schemes applicable to practical scenarios, and it is the reason the field grew from 2012 on.

Making it fast: packing and representations



Plaintext packing: from theory . . .

Let t be a prime which does not divide m , and let $\mathcal{R}_t = \mathbb{Z}_t[x]/\langle \Phi_m(x) \rangle$.

- $\Phi_m(x) \in \mathbb{Z}_t[x]$ **splits** mod t into κ **distinct irreducible factors** $F_i(x) \in \mathbb{Z}_t[x]$, each of the same degree N :

$$\Phi_m(x) = F_1(x) \cdots F_\kappa(x), \quad \deg(\Phi_m(x)) = \varphi(m) = N\kappa,$$

where N is the smallest positive integer such that $t^N \equiv 1 \pmod{m}$.

- By the **Chinese Remainder Theorem** for polynomials,

$$\begin{aligned} \mathcal{R}_t &\rightarrow \mathbb{Z}_t[x]/\langle F_1(x) \rangle \times \cdots \times \mathbb{Z}_t[x]/\langle F_\kappa(x) \rangle \approx \mathbb{F}_{t^N} \times \cdots \times \mathbb{F}_{t^N} \\ f(x) &\mapsto \left(f(x) \bmod F_1(x), \dots, f(x) \bmod F_\kappa(x) \right) \end{aligned}$$

Important for us

Each message in $\mathcal{R}_t$ actually corresponds to κ messages in $\mathbb{F}_{t^N}$. We call each of these components a **slot**.

...to an amortised cost

Special case: $m = 2^t = 2n$, $\Phi_m(x) = x^n + 1$ and t prime s.t. $t \equiv 1 \pmod{2n}$

$\implies \kappa = n$ and $N = 1$: letting ξ be a primitive m -th root of unity,

$$\mathcal{R}_t \approx \mathbb{Z}_t / \langle x - \xi_1 \rangle \times \dots \times \mathbb{Z}_t / \langle x - \xi_n \rangle,$$

that is, n slots, each one a plain element of $\mathbb{Z}_t$.

With n slots in one ciphertext, one homomorphic multiplication computes n multiplications at once: this is **SIMD** (single instruction, multiple data) on encrypted data.

With $n = 2^{15}$, a single ciphertext operation does 32768 useful ones.

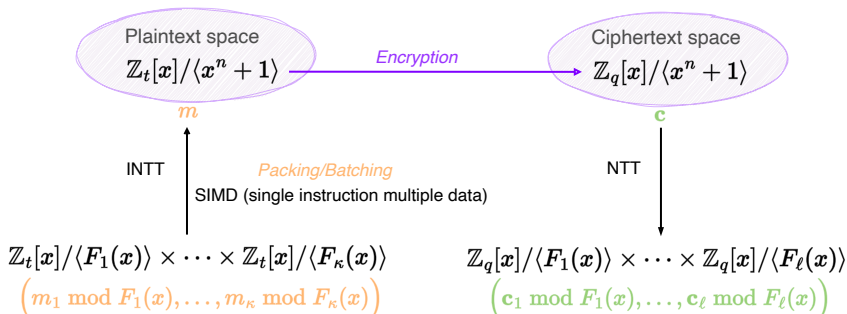
✓ the cost **per message** drops.

Number Theoretic Transform

The **Number Theoretic Transform (NTT)** and its inverse, the **INTT**, transform a polynomial into a point-wise representation such that

$$\text{INTT}(\text{NTT}(a) \odot \text{NTT}(b)) = a \cdot b$$

where $\odot$ denotes the **point-wise multiplication** of the transformed polynomials.



The last obstacle: arithmetic on huge numbers

We always work in $\mathcal{R}_q = \mathbb{Z}_q[x]/\langle x^n + 1 \rangle$, and a polynomial is stored by its coefficients $\mathbf{a} = [a_0, a_1, \dots, a_{n-1}] \in \mathbb{Z}_q^n$.

Problem 1: q is much bigger than the native word size (roughly 200 to 1500 bits), so we need multi-precision arithmetic $\implies$ very inefficient.

The last obstacle: arithmetic on huge numbers



We always work in $\mathcal{R}_q = \mathbb{Z}_q[x]/\langle x^n + 1 \rangle$, and a polynomial is stored by its coefficients $\mathbf{a} = [a_0, a_1, \dots, a_{n-1}] \in \mathbb{Z}_q^n$.

Problem 1: q is much bigger than the native word size (roughly 200 to 1500 bits), so we need multi-precision arithmetic $\implies$ very inefficient.

Problem 2: multiplying two polynomials of degree n costs n^2 coefficient multiplications, and n is 2^{15} or more.

The last obstacle: arithmetic on huge numbers

We always work in $\mathcal{R}_q = \mathbb{Z}_q[x]/\langle x^n + 1 \rangle$, and a polynomial is stored by its coefficients $\mathbf{a} = [a_0, a_1, \dots, a_{n-1}] \in \mathbb{Z}_q^n$.

Problem 1: q is much bigger than the native word size (roughly 200 to 1500 bits), so we need multi-precision arithmetic $\implies$ very inefficient.

Problem 2: multiplying two polynomials of degree n costs n^2 coefficient multiplications, and n is 2^{15} or more.

Two changes of representation solve both: RNS and NTT.

Solution P1: Residue Number System (RNS/CRT) representation

Let $B = \{p_0, p_1, \dots, p_\ell\}$ be a basis of pairwise coprime integers and let $q = \prod_{i=0}^{\ell} p_i$. We define

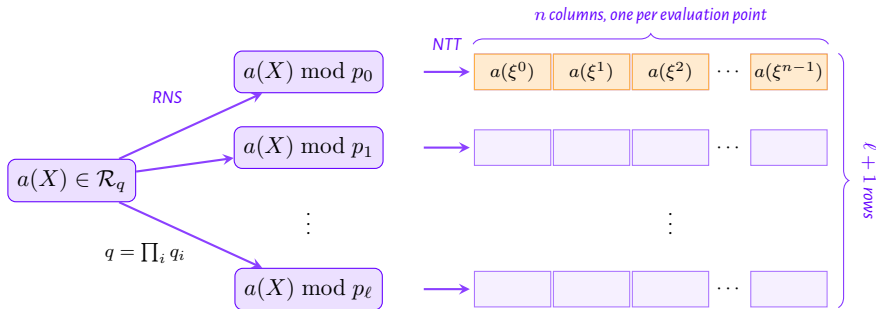
$$[\cdot]_B : \mathbb{Z}_q \longrightarrow \prod_{i=0}^{\ell} \mathbb{Z}_{p_i}, \quad a \mapsto [a]_B = ([a]_{p_i})_{0 \leq i \leq \ell}$$

- The mapping $[\cdot]_B$ is a ring isomorphism, by the **Chinese Remainder Theorem (CRT)**.
- This representation $[a]_B$ is known as the **Residue Number System (RNS)** representation of $a \in \mathbb{Z}_q$.
- RNS allows arithmetic operations to be performed **component-wise** in the smaller rings $\mathbb{Z}_{p_\ell}$, each of them of machine word size.
- The isomorphism extends naturally to polynomial rings: $[\cdot]_B : \mathcal{R}_q \longrightarrow \mathcal{R}_{p_0} \times \dots \times \mathcal{R}_{p_\ell}$

The chain of primes we introduced for the levels is exactly the RNS basis.

Double-CRT: how a polynomial is actually stored

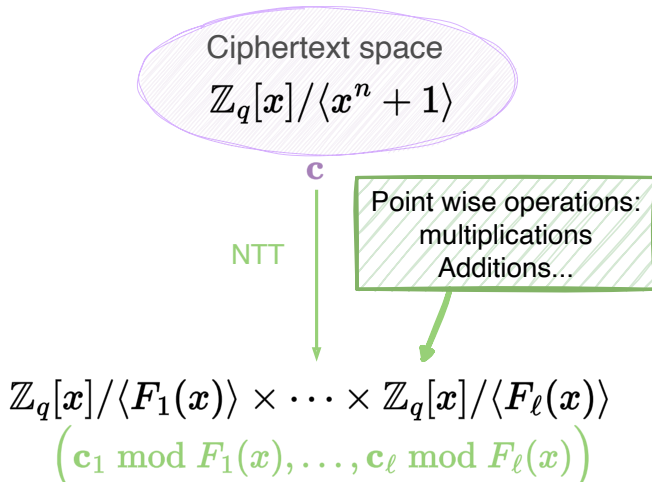
Put the two changes of representation together: **RNS** splits the modulus, the **NTT** splits the polynomial.



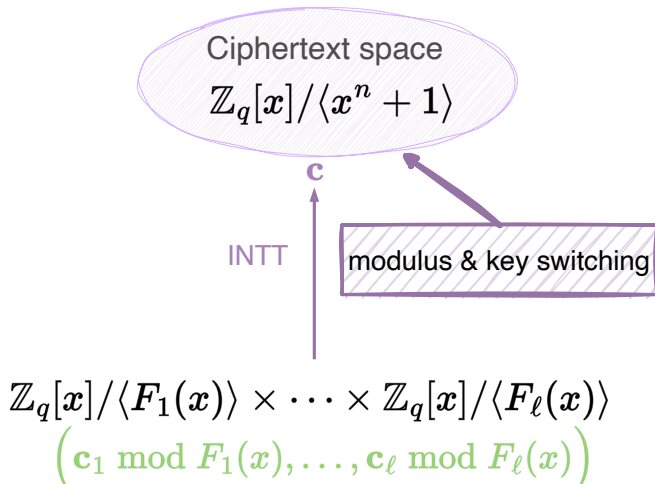
A polynomial is a $(\ell + 1) \times n$ matrix of machine words

Addition and multiplication are **entry-wise** on that matrix. With $n = 2^{15}$ and $\log q \approx 1500$ that is 25 rows of 32768 words, so one ciphertext (two polynomials) is about 12 MB.

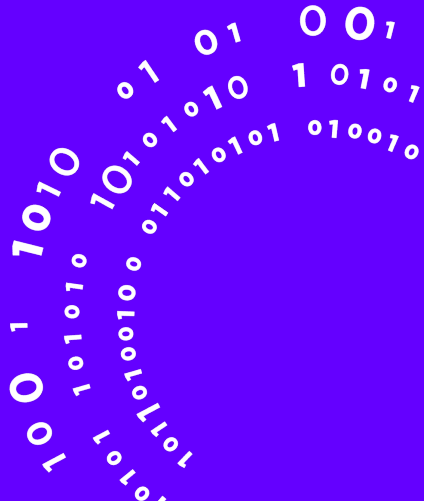
In (somehow) practice



In (somehow) practice

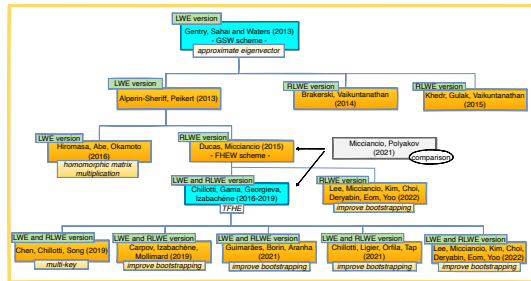


Back to history . . .

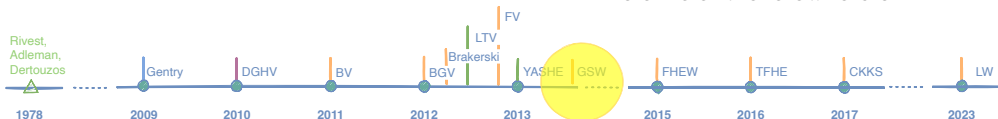


History... third generation

(R)LWE-based FHE schemes

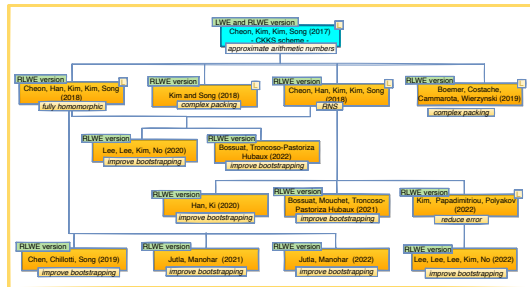


- + Approximate eigenvector: a new way to perform operations.
- + Fast bootstrapping: a new method to homomorphically compute the NAND (programmable bootstrapping)
- No packing technique
- No efficient leveled version

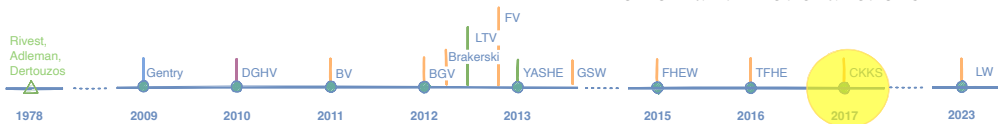


History... fourth generation

(R)LWE-based FHE schemes

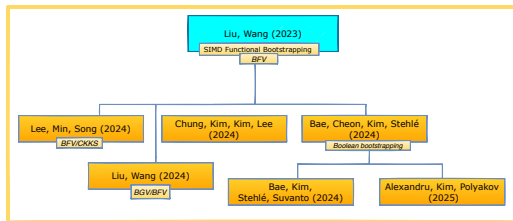


- + Approximate schemes
⇒ operations over (approximations of) real numbers.
- + Efficient packing techniques
- “Slow” bootstrapping (better than the second generation)
- No non-arithmetic functions

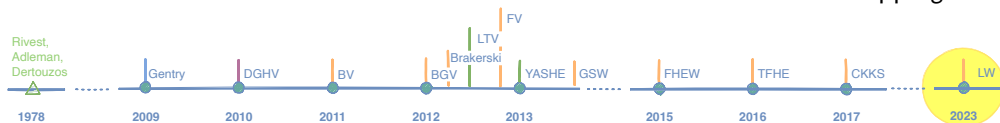


History... fifth generation

(R)LWE-based FHE schemes



- + Hybrid schemes
⇒ plaintext data represented as vectors of integers or Boolean values
- + Support the evaluation of arbitrary functions using look-up tables
- Slow functional bootstrapping



This is the part of the story that is still being written!

What functions can we evaluate homomorphically?

- BGV, BFV and CKKS are naturally limited to **multiplication**, **addition** and **subtraction** (in CKKS, also division by a constant) $\implies$ they work only on **arithmetic functions**.
- TFHE and LW use **programmable bootstrapping**: during the bootstrapping the scheme evaluates a look-up table, so the same operation that **reduces the noise** also evaluates **any function**.

Just to have an idea

In TFHE, functions are encoded by **storing discretized elements of their images**, and the evaluation selects a table position. This generalizes to any function, including non-polynomial ones, using the algorithm called **blind rotation**, which is part of the bootstrapping.

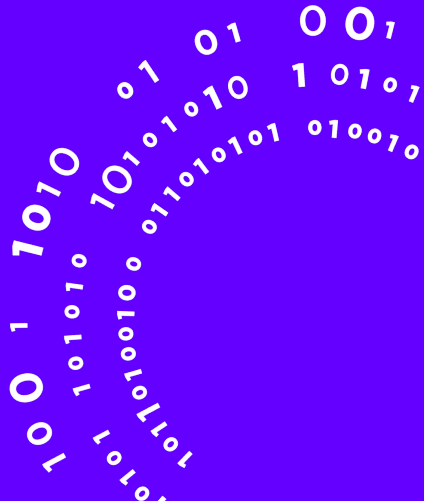
Backup – practical parameters

	BGV	BFV	CKKS	TFHE
arithmetic	integer		real number	bitwise operations
polynomial degree n	$2^{15} - 2^{17}$			$2^{10} - 2^{11}$
plaintext modulus t	better coprime with n $t \geq 2^{32} - 2^{64}$ $t \leq 2^{32} - 2^{64}$		—	2^κ
ciphertext modulus q	$2^{500} - 2^{1500}$			2^{32} or 2^{64}

Note. The practical crossover point is between $t = 2^{32}$ and $t = 2^{64}$ [1].

[1] Costache, Laine, Player - *Evaluating the effectiveness of heuristic worst-case noise analysis in FHE* (2020)

For tomorrow...



What we did not discuss

- **How much** the noise actually grows: every operation in this lecture has a noise bound, and those bounds are what decides whether a circuit is correct.
- **How to choose n, q, t and the moduli chain**: correctness, security and efficiency pull in opposite directions.
- **How secure** a given parameter set really is against lattice attacks.

This is exactly the subject of the second lecture.

**...Thank you for your
attention!**

